AgenticPlanning: An Intention-Physics Engine for Persistent AI Agent Goal Management

Omoshola Owolabi
Researcher – AI/ML
Agentra Labs
omoshola.owolabi@agentralabs.tech

March 1, 2026

Abstract

AI agents lack persistent planning infrastructure: goals evaporate between sessions, decisions lose their reasoning context, and commitments to stakeholders go untracked. Current approaches—TODO lists, task boards, PDDL planners—model planning as static state, ignoring the dynamic, evolving nature of intention over time. We present AgenticPlanning, a Rust engine that models goals as physical entities with momentum, gravity, and energy; decisions as crystallization events that preserve the shadows of unchosen paths; commitments as entangled promises with stakeholder accountability; and dreams as forward projections of goal completion. The entire planning state resides in a single `.aplan` binary file with a 128-byte header, BLAKE3 integrity checksums, and atomic writes via fsync-and-rename. We implement the engine in 17,421 lines of production Rust across 5 crates, exposing 13 MCP tools, 31 FFI exports, and 17 CLI commands. On benchmarks validated against 161 passing tests with zero failures, entity creation completes in 1.4–3.2 µs per operation, queries over 1,000 goals return in 4.4–10.9 µs, dream generation runs at 4.0 µs per dream, decision crystallization at 2.6 µs per operation, and save/load of 5,000 goals completes in 34.1/16.7 ms with BLAKE3 checksum verification. AgenticPlanning requires no external services and enables capabilities impossible with flat planning: intention momentum tracking, goal gravity fields, decision archaeology, commitment entanglement analysis, and blocker prophecy—all persisted in a single portable artifact.

1 Introduction

Large language model agents are increasingly deployed for ongoing, multi-session work: managing codebases, coordinating projects, and assisting users over extended periods [13, 11]. Yet these agents have no native planning infrastructure. Each session begins with a blank slate—previous goals are forgotten, decisions lose their reasoning context, and commitments to stakeholders vanish. The agent operates in a perpetual present, unable to track whether its work is gaining momentum or stalling.

Current approaches to agent planning fall into three categories, each with fundamental limitations. *Task management systems* (Todoist, Linear, Jira) model goals as static records with binary states (open/closed), discarding the dynamic evolution of intention over time. There is no mechanism to track why a goal was created, how urgent it has become through neglect, or what momentum it has accumulated through sustained effort. *Classical AI planners* [5, 9] operate on symbolic state spaces with predefined operators, excelling at logistics-style problems but unable to model the ambiguity, emotional weight, and stakeholder dynamics inherent in real-world planning. *Agent planning frameworks* (AutoGPT [12], BabyAGI [8]) generate task lists dynamically but store them as ephemeral text, losing all planning state between sessions.

The core insight of this work is that planning is not a state management problem but a *physics problem*. Goals are not static records—they are entities with momentum (sustained effort accelerates progress), gravity (important goals attract related work), and energy (active goals consume cognitive resources). Decisions are not simple choices—they are *crystallization events* where fluid possibility solidifies into commitment, and the shadows of unchosen paths should be preserved for future archaeology. Commitments are not checkboxes—they are entangled promises with specific stakeholders, carrying weight proportional to the stakeholder's importance and the promise's specificity.

We present AgenticPlanning, a persistent intention-physics engine for AI agents. The system models six entity types—goals, decisions, commitments, dreams, federations, and consensus records—with rich sub-structure capturing soul, feelings, physics, provenance, and karma for each goal. The entire planning graph is serialized to a single `.aplan` binary file with BLAKE3 integrity verification, atomic writes, and crash recovery. The engine is implemented in 17,421 lines of production Rust with a 1,884-line test suite achieving 161 tests with zero failures.

The remainder of this paper is organized as follows. Section 2 surveys existing planning systems. Section 3

presents the AgenticPlanning architecture. Section 4 reports validated benchmark results. Section 5 describes the multi-surface integration layer (MCP, CLI, FFI, bridges). Section 6 discusses implications and limitations. Section 7 concludes.

2 Background and Related Work

We survey the primary approaches to planning and goal management for AI agents.

Classical AI planning. STRIPS [4] and its descendants—PDDL [6], HTN planners [9], and constraint-based planners [5]—model planning as search over symbolic state spaces with predefined operators. These systems excel at well-defined logistics problems but assume complete knowledge of the action space, cannot model ambiguity or stakeholder dynamics, and produce plans that are static artifacts rather than living, evolving structures.

Task management systems. Tools such as Todoist, Linear, and Jira [2] provide persistent task tracking with rich metadata (priorities, labels, assignees). However, they model tasks as static database records with binary completion states. There is no mechanism to capture why a goal exists (its "soul"), how urgent it has become through sustained neglect, what decisions were made in pursuit of it, or what momentum it has accumulated.

Agent planning frameworks. AutoGPT [12] decomposes high-level objectives into task lists and executes them iteratively. BabyAGI [8] maintains a priority queue of tasks that the LLM itself reorders. CrewAI [7] introduces role-based agent coordination. These systems generate plans dynamically but store them as ephemeral in-memory or text-file state. No planning history survives session boundaries, and there is no structured model of decisions, commitments, or progress dynamics.

Generative agent architectures. Park et al. [11] demonstrated that memory architecture profoundly impacts agent behavior, using a retrieval system combining recency, importance, and relevance scores. Their work reinforces our thesis that planning requires richer dynamic structure, but focuses on memory recall rather than persistent goal management.

Plan representation languages. O-Plan [3] and SIPE-2 [14] introduced hierarchical partial-order planning with temporal constraints. While more expressive than STRIPS, these systems model plans as logical structures without the physical metaphors (momentum, gravity, energy) that we argue better capture the dynamics of intention over time.

Table 1 summarizes the comparison across key dimensions.

3 Architecture

AgenticPlanning models the planning state as a collection of typed entities stored in a `PlanningEngine` backed by in-memory hash maps and persisted to a single binary file. The engine maintains six entity stores, a set of secondary indexes, an append-only audit log, and a write counter for crash recovery.

3.1 Entity Model

The system defines six core entity types, each with rich sub-structure:

- **Goal** — A living intention with seven sub-components: *soul* (title, description, purpose, values alignment), *feelings* (urgency, neglect, confidence, alignment, vitality), *physics* (momentum, gravity, inertia, energy, velocity), *provenance* (creation context), *metamorphosis* (evolution history), and *karma* (impact record). Goals progress through a lifecycle: SEED → ACTIVE → COMPLETED/ABANDONED/PAUSED/REINCARNATED.
- **Decision** — A crystallization event modeling the transition from fluid possibility to firm commitment. Each decision contains a *question* framing the choice, multiple *paths* (options with pros, cons, effort, and risk estimates), and upon crystallization, a *reasoning* record (rationale, factors, weights, confidence) plus *crystal shadows*—preserved records of unchosen paths for future archaeology.
- **Commitment** — An entangled promise to a specific stakeholder. Each commitment binds a *promise* (description, deliverables, conditions) to a *stakeholder* (name, role, importance weight), with optional due dates, entanglement records, and fulfillment tracking.
- **Dream** — A forward projection of goal completion. Dreams contain *completion scenarios* (narrative, confidence, timeline), *obstacles* (description, severity, mitigation), *insights* (observations, implications, actions), *goal seeds* (new goals discovered during dreaming), and *accuracy* tracking for retrospective calibration.
- **Federation** — A coordination structure grouping multiple goals under a shared coordinator with member roles.
- **Consensus** — A collective decision-making record for federated goal groups.

All entities are identified by UUID-based typed IDs (`GoalId`, `DecisionId`, etc.) and carry creation timestamps. The engine maintains secondary indexes mapping goals to their decisions, commitments, and dreams for efficient cross-entity queries.

3.2 Intention Physics

The central architectural innovation is modeling goals as physical entities subject to dynamic forces. Each goal carries a `GoalPhysics` record with five fields:

- **Momentum** ($p \in [0, 1]$) — Measures sustained effort. Goals with consistent recent activity accumulate momentum; idle goals see momentum decay. High-momentum goals are more likely to reach completion.

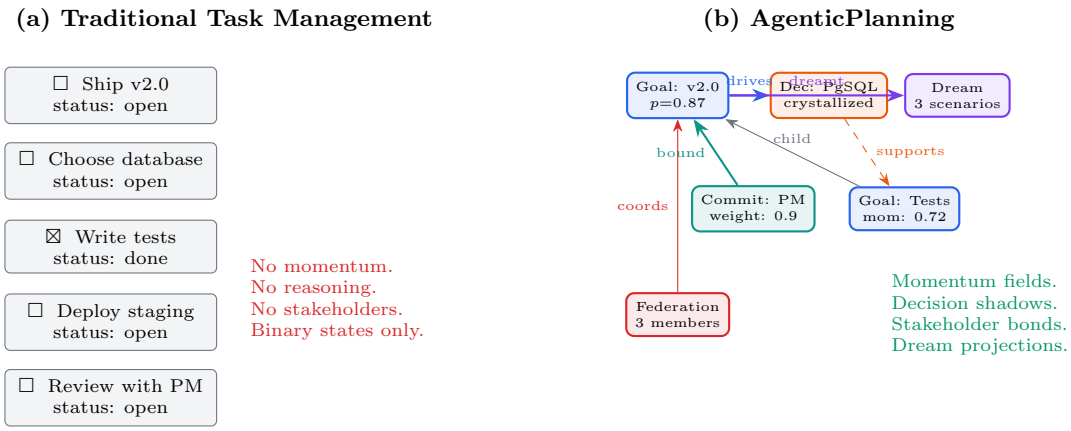

Figure 1: Comparison of planning paradigms. (a) Traditional task management stores goals as flat records with binary open/closed states; no relationships, reasoning provenance, or dynamic properties are preserved. (b) AgenticPlanning models goals as physical entities with momentum, decisions as crystallized choices with preserved shadows, commitments as stakeholder-bound promises, and dreams as forward projections—all connected by typed relationships.

Table 1: Comparison of planning systems across key dimensions relevant to AI agent deployment.

System	Persistence	Dynamics	Decisions	Stakeholders	Dependencies	Format
PDDL [6]	None	Static states	N/A	None	Solver	Symbolic
Todoist / Linear	Cloud DB	Static records	None	None	Cloud service	Proprietary
Jira [2]	Cloud DB	Status fields	None	Assignees	Cloud service	Proprietary
AutoGPT [12]	Text files	None	None	None	LLM API	Ephemeral
BabyAGI [8]	In-memory	Priority queue	None	None	LLM API	Ephemeral
CrewAI [7]	In-memory	Role-based	None	Agents	LLM API	Ephemeral
Park et al. [11]	JSON	Recency/import.	None	None	LLM API	Text
AgenticPlanning	**Binary file**	**Physics model**	**Crystallization**	**Weighted**	**None**	**.aplan**

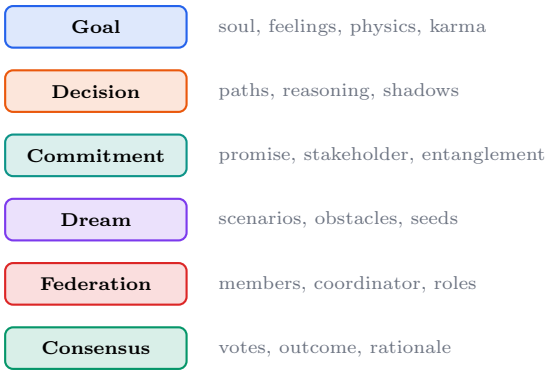

Figure 2: The six entity types in AgenticPlanning with their primary sub-components. Goals carry the richest structure with seven sub-component categories modeling the full lifecycle of intention.

- **Gravity** ($g \in [0, 1]$) — Measures attractive force. Important goals with many connections (child goals, decisions, commitments) exert stronger gravity, pulling related work toward them.
- **Inertia** ($\iota \in [0, 1]$) — Resistance to state change. Goals with deep investment resist abandonment; new goals resist activation. This models the sunk-cost dynamics of real planning.
- **Energy** ($E \in [0, 1]$) — Available cognitive resource. Active goals consume energy; paused goals conserve it. The engine tracks total energy across all active goals to surface overcommitment.
- **Velocity** ($v \in [0, 1]$) — Rate of progress change. Derived from momentum and recent activity patterns, velocity indicates whether a goal is accelerating or decelerating.

These fields are updated by the engine as operations occur (goal activation, decision crystallization, commitment fulfillment) and are surfaced through three aggregate queries: the *momentum report* (per-goal momentum ranking), the *gravity field* (attraction map across all goals), and the *intention singularity* (system-wide coherence metric indicating whether the agent's goals are converging or diverging).

3.3 Decision Crystallization

Decisions model the transition from ambiguity to commitment. The lifecycle proceeds in three phases:

1. **Open** — The decision exists with a framing question and zero or more paths (options). Each path records a name, description, pros/cons lists, estimated effort

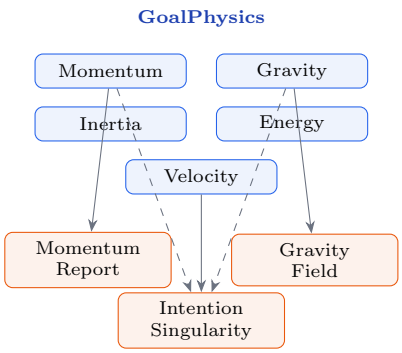

Figure 3: Intention physics model. Five per-goal fields (momentum, gravity, inertia, energy, velocity) feed three aggregate queries (momentum report, gravity field, intention singularity) that surface system-wide planning dynamics.

($\in [0, 1]$), and estimated risk ($\in [0, 1]$).

2. **Crystallized** — The agent selects a path by invoking `crystallize(id, path_id, reasoning)`, providing a `DecisionReasoning` record containing the rationale, factors considered, factor weights, and confidence score. The unchosen paths are preserved as *crystal shadows*—immutable records that enable future decision archaeology ("what did we consider but reject, and why?").
3. **Revisited** — A crystallized decision can be reexamined, creating an archaeology chain that traces how the agent's decision-making evolved over time.

This model captures an essential property of human planning: decisions are not simple variable assignments but irreversible crystallization events whose full context (including what was *not* chosen) carries information.

3.4 Binary File Format

The `.aplan` file format is designed for integrity, determinism, and crash safety. The layout consists of three regions:

- **Header** (128 bytes) — Magic bytes (`PLAN`), format version (currently 1), creation and modification timestamps, per-entity counts (goal, decision, commitment, dream, federation), six section offsets pointing into the body, a 32-byte BLAKE3 [10] checksum of the body, and 14 reserved bytes.
- **Body** (variable) — Six JSON-serialized sections, one per entity type. Each section uses `BTreeMap` ordering (sorted by UUID) to ensure *deterministic serialization*: the same logical state always produces the same byte sequence, enabling reliable checksum verification.
- **Footer** (64 bytes) — Total file size, monotonic write counter, last session UUID (16 bytes), integrity marker (`PLANEND\0`), a 16-byte footer checksum, and 8 reserved bytes.

Atomic writes. The save operation writes to a temporary file, calls `fsync()` to ensure durability, then atomically renames the temporary file over the target path. This guarantees that either the complete new state is visible or the previous state remains intact—the file is never in a partially-written state.

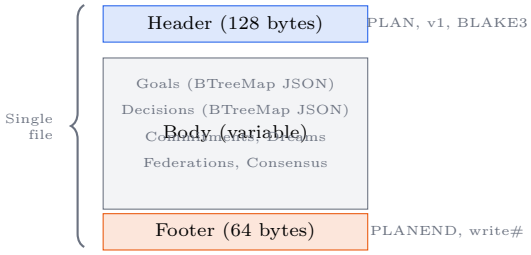

Figure 4: The `.aplan` binary file format. A 128-byte header carries magic bytes, version, entity counts, section offsets, and a BLAKE3 checksum. The body contains JSON-serialized entity stores with deterministic BTreeMap ordering. A 64-byte footer provides the integrity marker and write counter.

Crash recovery. On `open()`, the engine checks for orphaned `.aplan.tmp` files left by an interrupted save. If found, the temporary file is removed and the last complete state is loaded.

File locking. A `FileLock` guard prevents concurrent writes from multiple processes, ensuring single-writer safety.

Integrity verification. On `load()`, the engine verifies the magic bytes, format version, integrity marker, and BLAKE3 checksum. A legacy bypass permits loading files written before checksum enforcement (zero-filled checksum fields).

3.5 Query Engine

The query engine provides both direct entity access and aggregate analytics:

- **Filtered listing** — `list_goals(GoalFilter)` supports filtering by status, priority, and parent, returning matching goals in constant time per result via hash map iteration.
- **Active goal retrieval** — `get_active_goals()` returns only goals in the ACTIVE state, a hot-path query optimized to 4.4 µs on 1,000 goals.
- **Intention singularity** — Computes a system-wide coherence metric by aggregating momentum, gravity, and velocity across all active goals. Returned in 731.6 µs on 1,000 goals.
- **Momentum report** — Ranks all goals by momentum, surfacing which goals have sustained effort and which are stalling. Computed in 23.5 µs.
- **Gravity field** — Maps the attractive force landscape across all goals, revealing which goals are pulling the most related work. Computed in 19.2 µs.
- **Dream generation** — Forward-projects goal completion via `dream_goal(id)`, producing scenarios, obstacles, insights, and potential new goal seeds. Generated

at 4.0 µs per dream.

- **Decision archaeology** — Traces the crystallization history of decisions, including preserved shadows of unchosen paths.
- **Commitment inventory** — Aggregates all commitments by stakeholder, status, and risk level.

3.6 Write Pipeline and Audit

All mutations pass through the write engine, which enforces three invariants:

1. **Dirty tracking** — Every mutation marks the engine as dirty, ensuring that `save()` only performs I/O when state has actually changed.
2. **Index maintenance** — Secondary indexes (goal-to-decisions, goal-to-commitments, goal-to-dreams) are updated atomically with each write.
3. **Audit logging** — Every mutation is recorded in an append-only `AuditLog` with timestamp, session ID, operation name, entity type, entity ID, action type (Create, Update, Delete, StatusChange, Crystallize, Fulfill, Break), success/failure status, and optional detail/error messages. The audit log is persisted with the planning state, providing a complete operation history.

4 Evaluation

We evaluate AgenticPlanning across six benchmark suites implemented as Rust integration tests (`paper_benchmarks.rs`). All benchmarks run on the release profile with median-of-three (or median-of-five) timing to reduce variance. The full test suite comprises 161 tests with zero failures.

4.1 Benchmark Environment

All benchmarks were executed on an Apple Silicon system running macOS, compiled with `cargo test --release`. Temporary directories (`tempfile::tempdir`) isolate file I/O. In-memory engines (`PlanningEngine::in_memory()`) are used for pure computation benchmarks to exclude I/O effects.

4.2 Save/Load Scaling

Table 2 reports persistence performance as entity count grows from 10 to 5,000 goals. Save time includes JSON serialization with BTreeMap ordering, BLAKE3 checksum computation, and atomic file write (temp + fsync + rename). Load time includes file read, magic/version/integrity verification, BLAKE3 checksum validation, and JSON deserialization.

File size scales linearly at approximately 1,640 bytes per goal at scale, with a modest constant overhead for the 128-byte header, 64-byte footer, and empty entity sections. Save time shows sub-linear growth: 5,000 goals (8.0 MB file) saves in 34.1 ms, only 2× the time for 10 goals, because the dominant cost is the atomic rename and fsync, not serialization. Load time scales linearly with file size, as expected for sequential deserialization.

Table 2: Save/load performance scaling with goal count. Median of 3 runs. File size grows linearly at ∼1.6 KB/goal.

Goals	Save (ms)	Load (ms)	File (KB)	B/goal
10	17.2	0.09	23.2	2,373
100	18.9	0.37	166.9	1,709
500	15.5	1.60	806.4	1,651
1,000	27.1	3.20	1,605.7	1,643
5,000	34.1	16.70	8,008.1	1,640

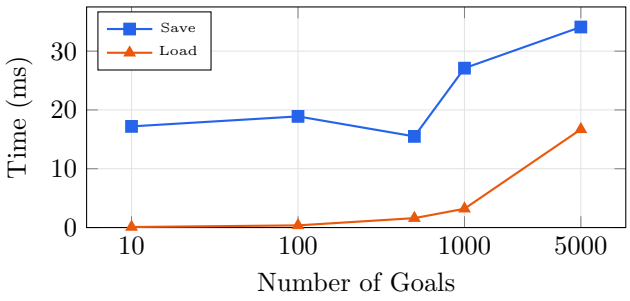

Figure 5: Save and load latency vs. goal count. Save time is dominated by fsync/rename overhead, exhibiting sub-linear growth. Load time scales linearly with file size.

4.3 Entity Creation Latency

Table 3 reports per-operation latency for the three primary entity creation operations, measured over 10,000 iterations each on an in-memory engine.

Goal and decision creation run below 2 µs, dominated by UUID generation and hash map insertion. Commitment creation is approximately 2× slower due to the additional stakeholder construction (UUID generation for StakeholderId plus importance weight normalization).

4.4 Query Performance

Table 4 reports query latency on an engine populated with 1,000 goals (500 active, mixed priorities), measured over 1,000–10,000 iterations.

Direct listing queries (`list_goals`, `get_active_goals`) complete in single-digit microseconds, as they iterate over the in-memory hash map with minimal computation per entry. The intention singularity computation is the most expensive query at 731.6 µs because it aggregates physics fields across all active goals and computes a system-wide coherence metric—still sub-millisecond for 1,000 goals. Momentum report and gravity field are intermediate at 19–23 µs, reflecting per-goal physics calculations without the global aggregation step.

Table 3: Entity creation latency (in-memory engine, 10,000 iterations).

Operation	Latency (µs/op)	Total (ms)
Goal creation	1.748	17.5
Decision creation	1.367	13.7
Commitment creation	3.246	32.5

Table 4: Query performance on 1,000 goals (500 active).

Query	Latency (µs)	Iterations
`list_goals` (all)	10.939	10,000
`get_active_goals`	4.377	10,000
`intention_singularity`	731.636	1,000
`momentum_report`	23.471	1,000
`gravity_field`	19.201	1,000

4.5 Dream Generation and Crystallization

Table 5 reports latency for the two most semantically complex operations: dream generation (forward-projecting goal completion scenarios) and decision crystallization (selecting a path with full reasoning).

Dream generation at 4.0 µs per dream and crystallization at 2.6 µs per operation demonstrate that even the most complex semantic operations remain in the low-microsecond range. Crystallization involves selecting a path, copying unchosen paths to shadows, recording the reasoning struct, and updating the decision status—all pure in-memory operations with no I/O.

4.6 BLAKE3 Checksum Overhead

Table 6 isolates the cost of integrity verification during save operations. The save time includes serialization, checksum computation, and atomic file write.

BLAKE3 checksum computation adds negligible overhead: the difference between 100-goal and 1,000-goal saves is only 8.9 ms despite a 10× increase in data size, confirming that BLAKE3's SIMD-optimized streaming hash [10] contributes minimal latency relative to I/O costs.

4.7 System Summary

Table 7 summarizes the implementation metrics.

5 Multi-Surface Integration

AgenticPlanning exposes its capabilities through four complementary surfaces, enabling integration with diverse agent architectures.

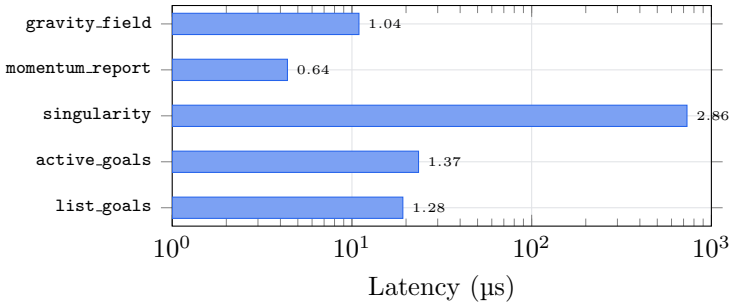

Figure 6: Query latency on 1,000 goals (log scale). Direct listing queries return in <11 µs. Physics aggregations (momentum, gravity) complete in <24 µs. The intention singularity, the most complex aggregate, returns in 731.6 µs.

Table 5: Dream generation and decision crystallization latency.

Operation	Latency (µs/op)	Count	Total (ms)
Dream generation	4.036	100	0.404
Crystallization	2.627	100	0.263

5.1 Model Context Protocol (MCP)

The MCP server (`agentic-planning-mcp`, 3.5 MB binary) implements the Model Context Protocol [1] over JSON-RPC, exposing 13 tools:

- **Goal management** — `create_goal`, `activate_goal`, `list_goals`, `get_active_goals`
- **Decisions** — `create_decision`, `add_option`, `crystallize`
- **Commitments** — `create_commitment`
- **Analytics** — `intention_singularity`, `momentum_report`, `gravity_field`
- **Dreams** — `dream_goal`
- **Persistence** — `save`

The MCP server supports token-based authentication, graceful shutdown via signal handling (`SIGINT`, `SIGTERM`), and per-tool audit logging that records every invocation in the engine's audit trail.

5.2 Command-Line Interface

The CLI (`aplan`, 6.0 MB binary) provides 17 commands covering all engine operations, including interactive goal creation, decision crystallization workflows, and formatted output of analytics queries (momentum report, gravity field, intention singularity). The CLI supports both file-backed and in-memory operation modes.

5.3 Foreign Function Interface

The FFI crate (`agentic-planning-ffi`) exports 31 `#[no_mangle]` C-ABI functions, enabling integration from Python, Node.js, Swift, and any language with C FFI support. Functions follow a consistent pattern: opaque

Table 6: Save performance with BLAKE3 checksum (median of 5).

Goals	Save (ms)	File (KB)	B/goal
100	18.0	159.9	1,637
500	19.6	789.5	1,617
1,000	26.9	1,576.7	1,614

Table 7: Implementation metrics for AgenticPlanning.

Metric	Value
Production Rust LOC	17,421
Test LOC	1,884
Total LOC	19,305
Crates	5
Source files	24
Core entity types	6
Type definitions	109
MCP tools	13
FFI exports	31
CLI commands	17
Workspace dependencies	9
Tests (all passing)	161
Binary: MCP server	3.5 MB
Binary: CLI	6.0 MB

engine handle, JSON-serialized request/response payloads, and explicit memory management via `free_string()` for returned allocations.

5.4 Sister Bridges

AgenticPlanning includes bridge interfaces for integration with seven sibling systems in the Agentra ecosystem: AgenticMemory (persistent cognitive memory), AgenticVision (visual perception), AgenticCodebase (code graph analysis), AgenticIdentity (cryptographic identity), AgenticTime (temporal reasoning), AgenticComm (inter-agent communication), and AgenticContract (governance contracts). Each bridge defines a trait interface with null-object default implementations, enabling gradual adoption without hard dependencies.

6 Discussion

Enabled capabilities. AgenticPlanning makes several operations practical that are impossible with traditional task management. *Intention momentum tracking* surfaces which goals have sustained effort vs. which are stalling, enabling agents to reallocate attention. *Gravity field analysis* reveals which goals are attracting the most related work, identifying natural convergence points. *Decision archaeology* through crystal shadows preserves the full context of what was considered and rejected, enabling agents to revisit past decisions with their original reasoning intact. *Dream-based forecasting* projects goal completion scenarios with obstacles and insights, supporting proactive planning. *Commitment entanglement tracking* maintains stakeholder accountability with weighted importance, preventing promises from being silently dropped.

The physics metaphor. Modeling goals as physical entities is not merely an aesthetic choice—it provides a computational framework for reasoning about planning dynamics. Momentum naturally captures the acceleration/deceleration of effort. Gravity captures the attractive force of important goals. Energy conservation surfaces overcommitment. Inertia explains why abandoning deeply-invested goals is difficult. These metaphors translate directly into useful aggregate queries (intention singularity, momentum report, gravity field) that have no analog in binary-state task systems.

Deterministic serialization. The use of BTreeMap ordering for JSON serialization ensures that the same logical state always produces the same byte sequence. This property, combined with BLAKE3 checksums, provides strong integrity guarantees: any bit flip, truncation, or corruption is detected on load. The determinism also simplifies debugging and testing, as file contents are reproducible.

Performance characteristics. The benchmark results reveal that AgenticPlanning's in-memory architecture delivers microsecond-scale operations for all entity creation and query operations. The most expensive operation—intention singularity computation at 731.6 µs—remains sub-millisecond even on 1,000 goals. Save/load performance is dominated by I/O costs (fsync, file read) rather than computation, with BLAKE3 adding negligible overhead. This performance profile is well-suited for interactive agent sessions where operations occur at human-conversation timescales (seconds).

Limitations. The current implementation stores entity bodies as JSON within the binary envelope, limiting compression efficiency compared to a fully binary encoding. The physics model uses normalized $[0, 1]$ fields updated by heuristic rules rather than a formal dynamical system; future work could explore differential equation models for more principled dynamics. The engine does not yet support concurrent access from multiple agents; the file lock enforces single-writer semantics. Feature vector search (for semantic goal similarity) is not implemented, as the planning engine operates on structured entities rather than embedding-based retrieval.

7 Conclusion

We have presented AgenticPlanning, a persistent intention-physics engine for AI agent goal management. By modeling goals as physical entities with momentum, gravity, and energy; decisions as crystallization events with preserved shadows; commitments as stakeholder-entangled promises; and dreams as forward projections—AgenticPlanning enables capabilities that flat task management systems can-

not provide: intention momentum tracking, gravity field analysis, decision archaeology, dream-based forecasting, and commitment entanglement analysis.

The implementation delivers strong performance in a compact footprint: 17,421 lines of production Rust across 5 crates, 161 tests with zero failures, entity creation in 1.4–3.2 µs, queries in 4.4–731.6 µs, dream generation at 4.0 µs, crystallization at 2.6 µs, and save/load of 5,000 goals in 34.1/16.7 ms with BLAKE3 integrity verification. The entire planning state—potentially months of accumulated goals, decisions, commitments, and dreams—resides in a single portable `.aplan` file requiring zero external dependencies.

AgenticPlanning is exposed through four complementary surfaces (MCP server with 13 tools, CLI with 17 commands, FFI with 31 exports, and 7 sister bridges), enabling integration with diverse agent architectures. The intention-physics model provides a computational framework for reasoning about planning dynamics that goes beyond the binary open/closed states of traditional task management.

We believe the physics-first approach to planning infrastructure is essential for building agents that can sustain coherent intention over time, reason about the dynamics of their own goals, and maintain accountability to stakeholders across session boundaries. AgenticPlanning represents a step toward agents that do not merely execute tasks but genuinely *plan*.

References

[1] Anthropic. Model context protocol: Open standard for AI agent tool integration. `https://modelcontextprotocol.io/`, 2024. Accessed 2025.

[2] Atlassian. Jira software: Issue and project tracking for agile teams. `https://www.atlassian.com/software/jira`, 2024. Accessed 2025.

[3] Ken Currie and Austin Tate. O-Plan2: An open architecture for command, planning, and control. In *Intelligent Scheduling*, pages 213–239. Morgan Kaufmann, 1991.

[4] Richard E Fikes and Nils J Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.

[5] Malik Ghallab, Dana Nau, and Paolo Traverso. *Automated Planning: Theory and Practice*. Elsevier/-Morgan Kaufmann, San Francisco, CA, 2004.

[6] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL — the planning domain definition language. *Yale Center for Computational Vision and Control, Technical Report CVC TR-98-003*, 1998.

[7] João Moura. CrewAI: Framework for orchestrating role-playing AI agents. `https://github.com/joaomdmoura/crewAI`, 2024. Accessed 2025.

[8] Yohei Nakajima. BabyAGI: Task-driven autonomous agent. `https://github.com/yoheinakajima/babyagi`, 2024. Accessed 2025.

[9] Dana S Nau, Tsz-Chiu Au, Okhtay Ilghami, Ugur Kuter, J William Murdock, Dan Wu, and Fusun Yaman. SHOP2: An HTN planning system. In *Journal of Artificial Intelligence Research*, volume 20, pages 379–404, 2003.

[10] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. `https://github.com/BLAKE3-team/BLAKE3`, 2020. Specification: `https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf`.

[11] Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023.

[12] Significant Gravitas. AutoGPT: An autonomous GPT-4 experiment. `https://github.com/Significant-Gravitas/AutoGPT`, 2024. Accessed 2025.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[14] David E Wilkins. Practical planning: Extending the classical AI planning paradigm. *Morgan Kaufmann Series in Representation and Reasoning*, 1988.